Kalman and Particle Filtering of the Quadruple-Tank Process

Sidharth Subramaniyan (ED24B066)

Code: https://github.com/SendCodePlz/Estimation_And_Control_Mini_Project

Estimation and Control — Mini Project

Abstract

We estimate the four water levels of Johansson's quadruple-tank process when only the two lower tanks are measured. A linear Kalman filter (KF), built on the linearized model, and a bootstrap particle filter (PF), built on the full non-linear model, are applied to 1000 s of simulated data. Both recover the measured levels h_1, h_2 about 2.6× more accurately than the raw sensors (RMSE 0.038 cm vs 0.100 cm) and estimate the unmeasured levels h_3, h_4 to within about 0.16–0.20 cm. We derive the covariance relations that explain why the unmeasured tanks are estimated so much less precisely, show how the prior and posterior covariances expand and contract every step, validate both filters with a $\pm 2\sigma$ consistency test, study the effect of tuning Q and R, and explain the mathematics and practical behaviour of the particle filter, including resampling and the choice of particle count.

Contents

How the work was done **2**

Notation **2**

1 System dynamics **2**
1.1 Physical set-up . . . 2
1.2 Non-linear model . . . 3
1.3 Equilibrium and linearization . . . 4
1.4 Discrete-time model . . . 4
1.5 Observability check . . . 4

2 Data generation **5**

3 The Kalman filter **6**
3.1 Algorithm . . . 6
3.2 How the covariance terms are related . . . 7

4 Kalman filter results **8**
4.1 Prior and posterior: the covariance ellipse expands and contracts . . . 8
4.2 How P_{11} converges from a poor initialization . . . 9
4.3 Convergence of the Kalman gain . . . 10
4.4 Validation of the decoupled structure . . . 11
4.5 Filtered estimates . . . 11
4.6 Validation with the $\pm 2\sigma$ band . . . 12

5 Tuning the Kalman filter **13**

6 The particle filter: theory **15**
6.1 Notation . . . 15
6.2 The exact Bayesian filter . . . 15
6.3 Monte Carlo approximation . . . 16
6.4 Importance sampling and the weight update . . . 16
6.5 Degeneracy and resampling . . . 17
6.6 Choosing the number of particles . . . 18

7 Particle filter results **18**
7.1 Estimates and the particle cloud . . . 18
7.2 Resampling in practice . . . 20
7.3 Validation and number of particles . . . 21
7.4 Kalman filter vs particle filter . . . 23

8 Conclusions **24**

How the work was done

Two sets of code accompany this report, written for different purposes.

- **MATLAB** (`Data_Generation.m`, `Kalman_Filter.m`, `Particle_Filter.m`) is the working implementation. It was used to generate the dataset, to implement the Kalman and particle filters step by step, and to experiment with them (tuning Q and R, changing the initial guess, the number of particles and the resampling threshold) in order to understand how each part of the filters behaves. Each filter script has a tuning block at the top, where the filter's Q and R are set as multiples of the true noise levels, and saves its results to a `.mat` file. The scripts are included in the `matlab/` folder of the code.
- **Python** (the notebook `filters_report.ipynb`) re-implements the same model and filters on the same dataset (its Kalman filter matches the MATLAB one to rounding error) in order to produce the many figures and tables in this report in one reproducible run. It writes every figure to `figures/` and every table to `tables/`, which this document includes directly, so re-running the notebook and recompiling updates the report.

Notation

Symbol	Meaning
$x_k = [h_1, h_2, h_3, h_4]^\mathsf{T}$	true tank levels at time step k (cm); $t = k\Delta t$, $\Delta t = 0.1\,\text{s}$, $N = 10\,000$ steps
$u_k = [v_1, v_2]^\mathsf{T}$	pump voltages (V), the known inputs
y_k	sensor readings of tanks 1 and 2 (V)
$w_k \sim \mathcal{N}(0, Q)$	process noise: unmodelled disturbances that really change the levels
$\nu_k \sim \mathcal{N}(0, R)$	measurement noise: sensor error that changes only the reading
F, G, C	discrete-time state, input and output matrices; k_c is the sensor gain
x^0, u^0	operating point (equilibrium) used for linearization
$\hat{x}_k^-$	*prior* (predicted) estimate: before the measurement y_k is used
$\hat{x}_k^+$	*posterior* (updated) estimate: after the measurement y_k is used
$e_k^\pm = x_k - \hat{x}_k^\pm$	estimation error
$P_k^\pm = E[e_k^\pm e_k^{\pm\mathsf{T}}]$	error covariance; P_{ij} is its (i,j) entry, $\sigma_i = \sqrt{P_{ii}}$, $\rho_{ij} = P_{ij}/\sqrt{P_{ii}P_{jj}}$
$r_k = y_k - C\hat{x}_k^-$	*innovation*: how much the measurement surprises the filter
$S_k = CP_k^-C^\mathsf{T} + R$	*innovation covariance*: how large the innovation is expected to be; $CP_k^-C^\mathsf{T}$ comes from the uncertainty of the state, R from the sensor noise
$K_k = P_k^-C^\mathsf{T}S_k^{-1}$	Kalman gain: how much of the innovation is used to correct each state
P_∞, K_∞	steady-state covariance and gain

Particle-filter symbols are listed separately in Section 6.1.

1 System dynamics

1.1 Physical set-up

Two pumps feed four tanks (Figure 1). Each pump's flow is split by a valve: pump 1 sends a fraction γ_1 to tank 1 and $1-\gamma_1$ to tank 4; pump 2 sends γ_2 to tank 2 and $1-\gamma_2$ to tank 3. The upper tanks drain into the lower ones (tank 3 into tank 1, tank 4 into tank 2), and the lower tanks drain into a reservoir. Only the levels of tanks 1 and 2 are measured.

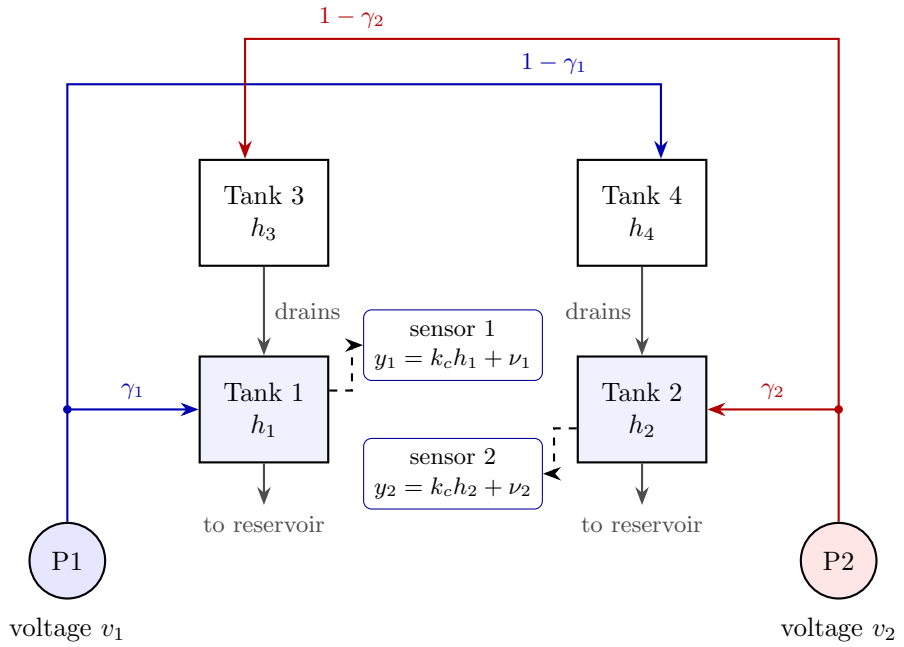


Figure 1: The quadruple-tank process. Pump 1 feeds tanks 1 and 4, pump 2 feeds tanks 2 and 3. Only tanks 1 and 2 (shaded) are measured; tanks 3 and 4 are seen only through what drains into the tanks below.

1.2 Non-linear model

Each tank obeys a mass balance, area × rate of change = inflow − outflow, with outflow through the bottom hole given by Torricelli's law, $a_i\sqrt{2gh_i}$:

$$\begin{aligned} \dot{h}_1 &= -\frac{a_1}{A_1}\sqrt{2gh_1} + \frac{a_3}{A_1}\sqrt{2gh_3} + \frac{\gamma_1 k_1}{A_1}v_1, & \dot{h}_2 &= -\frac{a_2}{A_2}\sqrt{2gh_2} + \frac{a_4}{A_2}\sqrt{2gh_4} + \frac{\gamma_2 k_2}{A_2}v_2, \\ \dot{h}_3 &= -\frac{a_3}{A_3}\sqrt{2gh_3} + \frac{(1-\gamma_2)k_2}{A_3}v_2, & \dot{h}_4 &= -\frac{a_4}{A_4}\sqrt{2gh_4} + \frac{(1-\gamma_1)k_1}{A_4}v_1. \end{aligned} \tag{1}$$

The state is $x = [h_1, h_2, h_3, h_4]^\mathsf{T}$ (cm), the input $u = [v_1, v_2]^\mathsf{T}$ (V) and the output $y = [k_c h_1,\ k_c h_2]^\mathsf{T}$ (V). The square root is the only non-linearity: a fuller tank drains faster. Table 1 lists the parameters.

Quantity	Symbol	Value	Unit
Tank cross-sections	A_1, A_2, A_3, A_4	28, 32, 28, 32	cm^2
Outlet areas	a_1, a_2, a_3, a_4	0.071, 0.057, 0.071, 0.057	cm^2
Valve splits	γ_1, γ_2	0.7, 0.6	–
Pump gains	k_1, k_2	3.33, 3.35	$\mathrm{cm}^3/(\mathrm{V\,s})$
Sensor gain	k_c	0.5	V/cm
Gravity	g	981	$\mathrm{cm/s}^2$
Pump voltages	v_1, v_2	3, 3	V
Initial levels	$h(0)$	12.4, 12.7, 1.8, 1.4	cm

Table 1: Process parameters (Johansson, 2000).

Reading the equations for *who feeds whom* gives the key structural facts:

- h_3 appears in the $\dot{h}_1$ equation, but h_1 never appears in the $\dot{h}_3$ equation: water flows down, never up. The same holds for h_4 and h_2.
- The pair (h_1, h_3) shares no pipe with the pair (h_2, h_4). The pumps feed both pairs, but the pump voltages are known, so they create no uncertainty link between the pairs.

1.3 Equilibrium and linearization

Setting every derivative in (1) to zero with $v_1 = v_2 = 3\,\mathrm{V}$ gives the equilibrium

$$h_3^0 = \frac{1}{2g}\Big(\frac{(1-\gamma_2)k_2 v_2}{a_3}\Big)^2, \qquad h_1^0 = \frac{1}{2g}\Big(\frac{\gamma_1 k_1 v_1 + a_3\sqrt{2gh_3^0}}{a_1}\Big)^2, \tag{2}$$

with the analogous expressions for h_4^0, h_2^0, giving $x^0 = [12.26,\ 12.78,\ 1.63,\ 1.41]^\mathsf{T}$ cm. Linearizing (1) about (x^0, u^0) in deviation variables $\Delta x = x - x^0$, $\Delta u = u - u^0$ gives (Slide 3)

$$\Delta\dot{x} = A_c\,\Delta x + B_c\,\Delta u, \tag{3}$$

$$A_c = \begin{bmatrix} -\frac{1}{T_1} & 0 & \frac{A_3}{A_1 T_3} & 0 \\ 0 & -\frac{1}{T_2} & 0 & \frac{A_4}{A_2 T_4} \\ 0 & 0 & -\frac{1}{T_3} & 0 \\ 0 & 0 & 0 & -\frac{1}{T_4} \end{bmatrix}, \quad B_c = \begin{bmatrix} \frac{\gamma_1 k_1}{A_1} & 0 \\ 0 & \frac{\gamma_2 k_2}{A_2} \\ 0 & \frac{(1-\gamma_2)k_2}{A_3} \\ \frac{(1-\gamma_1)k_1}{A_4} & 0 \end{bmatrix}, \tag{4}$$

where $T_i = (A_i/a_i)\sqrt{2h_i^0/g}$ is the time a tank takes to drain back towards equilibrium: $T_1 = 62.4$, $T_2 = 90.6$, $T_3 = 22.8$ and $T_4 = 30.1\,\mathrm{s}$. Only h_1 and h_2 are measured, so

$$C = \begin{bmatrix} k_c & 0 & 0 & 0 \\ 0 & k_c & 0 & 0 \end{bmatrix}. \tag{5}$$

1.4 Discrete-time model

With the forward difference over $\Delta t = 0.1\,\mathrm{s}$, $F = I + A_c\Delta t$ and $G = B_c\Delta t$:

$$F \approx \begin{bmatrix} 0.99840 & 0 & \mathbf{0.00439} & 0 \\ 0 & 0.99890 & 0 & \mathbf{0.00332} \\ 0 & 0 & 0.99561 & 0 \\ 0 & 0 & 0 & 0.99668 \end{bmatrix}. \tag{6}$$

Two numbers drive everything that follows. $F_{11} \approx 0.998$ says that tank 1 keeps almost all of its level from one step to the next. $F_{13} \approx 0.0044$ says that each step tank 1 picks up only 0.44% of tank 3's deviation. This is the only path by which h_3 reaches a measurement, and it is thin.

1.5 Observability check

Before building any filter we must check that the sensors carry enough information to recover *all four* levels, even though only two are measured. For a linear discrete-time system this holds if and only if the observability matrix has full rank:

$$\mathcal{O} = \begin{bmatrix} C \\ CF \\ CF^2 \\ CF^3 \end{bmatrix} = \begin{bmatrix} 0.5 & 0 & 0 & 0 \\ 0 & 0.5 & 0 & 0 \\ 0.4992 & 0 & \mathbf{0.0022} & 0 \\ 0 & 0.4995 & 0 & \mathbf{0.0017} \\ 0.4984 & 0 & \mathbf{0.0044} & 0 \\ 0 & 0.4989 & 0 & \mathbf{0.0033} \\ 0.4976 & 0 & \mathbf{0.0066} & 0 \\ 0 & 0.4984 & 0 & \mathbf{0.0050} \end{bmatrix}. \tag{7}$$

The first two rows (C) see only h_1, h_2. The h_3, h_4 columns become non-zero only from the second block on: y_1 one step later contains $k_c F_{13} h_3$, because tank 3 has drained into tank 1 in the meantime.

Result: the system is observable, but h_3 and h_4 only weakly.

- $\operatorname{rank} \mathcal{O} = 4$: all four levels can be recovered from y_1, y_2.
- Without the couplings ($F_{13} = F_{24} = 0$) the rank drops to 2: h_3, h_4 would be invisible.
- The singular values of $\mathcal{O}$ are 0.998, 0.998, 0.0049, 0.0037. The two directions along h_3, h_4 are about 270× weaker than those along h_1, h_2 (condition number 269).

Full rank says a filter *can* estimate h_3 and h_4; the tiny singular values say it will do so much less precisely than h_1, h_2. This single fact explains most of the results that follow.

2 Data generation

The dataset (Q-1) is produced by integrating the *non-linear* model (1) with the Euler–Maruyama scheme and adding Gaussian noise (Figure 2):

$$x_{k+1} = \max\bigl(0,\; x_k + \Delta t\, f(x_k, u_k) + w_k\bigr), \quad w_k \sim \mathcal{N}(0, Q), \qquad y_k = C x_k + \nu_k, \quad \nu_k \sim \mathcal{N}(0, R), \tag{8}$$

with $\sigma_w = 0.05\,\mathrm{cm}/\sqrt{\mathrm{s}}$, $Q = \sigma_w^2 \Delta t\, I = 2.5 \times 10^{-4} I\,\mathrm{cm}^2$ per step, and sensor noise of standard deviation $\sigma_v = 0.05\,\mathrm{V}$, $R = 2.5 \times 10^{-3} I\,\mathrm{V}^2$ (0.1 cm of level). The variance of w_k scales with Δt because independent kicks add up like a random walk; this keeps the physical noise level the same for any step size. $N = 10\,000$ samples cover 1000 s, about ten time constants of the slowest tank. The step $\Delta t = 0.1\,\mathrm{s}$ is about 230× smaller than the fastest time constant, so the Euler error is negligible (it differs from an `ode45` solution by about 10^{-4} cm).

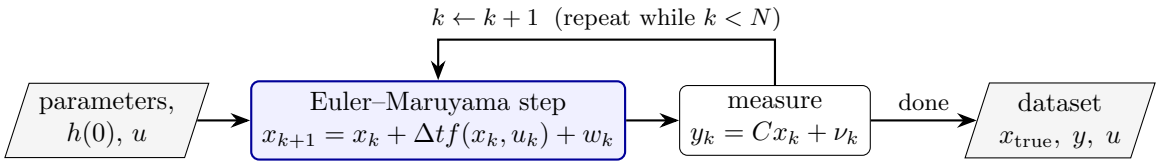


Figure 2: Data generation. The true levels x_{true} are kept only for validating the filters; the filters see only y and u.

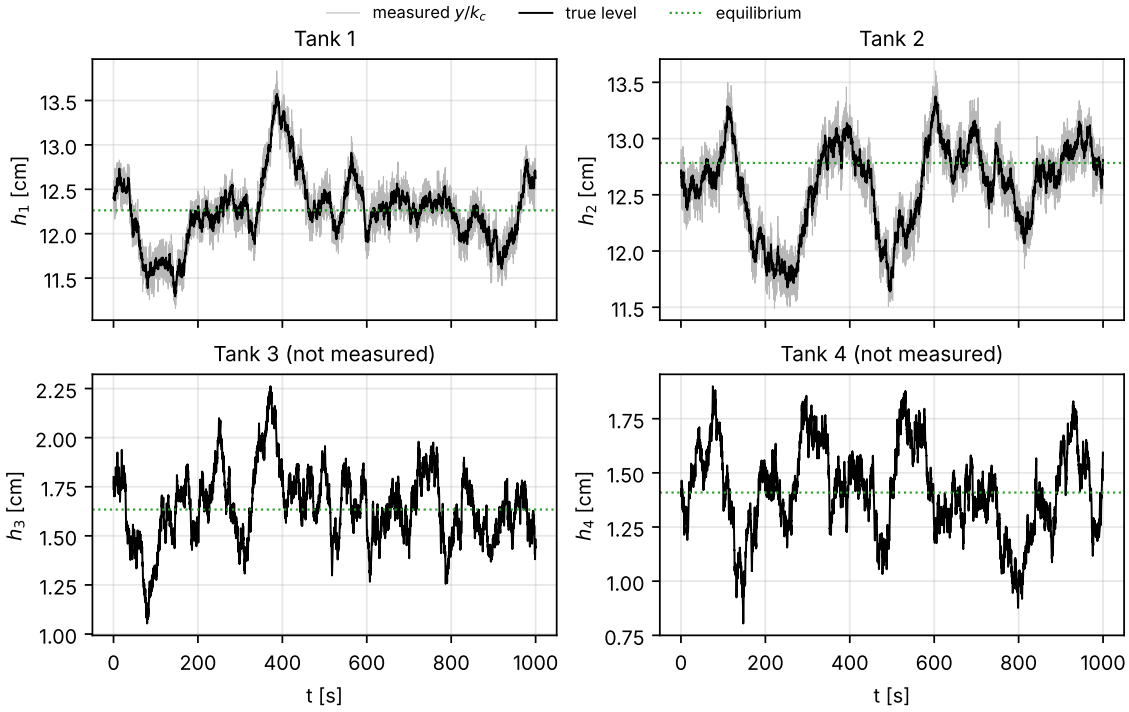


Figure 3: The generated data. The process noise makes the true levels (black) wander around the equilibrium (dotted). Measurements (grey) exist only for tanks 1 and 2.

3 The Kalman filter

3.1 Algorithm

The KF uses the linear model $\Delta x_{k+1} = F\Delta x_k + G\Delta u_k + w_k$, $y_k = Cx_k + \nu_k$ with Gaussian w_k, ν_k, and keeps a Gaussian belief $\mathcal{N}(\hat{x}_k, P_k)$ about the state. Each step alternates a prediction and a measurement update (Figure 4):

$$\text{predict:}\quad \hat{x}_k^- = x^0 + F(\hat{x}_{k-1}^+ - x^0) + G(u_{k-1} - u^0), \quad P_k^- = FP_{k-1}^+F^\mathsf{T} + Q, \tag{9}$$

$$\text{gain:}\quad S_k = CP_k^-C^\mathsf{T} + R, \quad K_k = P_k^-C^\mathsf{T}S_k^{-1}, \tag{10}$$

$$\text{update:}\quad \hat{x}_k^+ = \hat{x}_k^- + K_k\,(y_k - C\hat{x}_k^-), \quad P_k^+ = (I - K_kC)P_k^-(I - K_kC)^\mathsf{T} + K_kRK_k^\mathsf{T}. \tag{11}$$

Here $r_k = y_k - C\hat{x}_k^-$ is the innovation and S_k is its covariance: $CP_k^-C^\mathsf{T}$ is the spread of the predicted measurement caused by the uncertainty in the state, and R is the spread added by the sensor noise. The gain $K_k = P_k^-C^\mathsf{T}S_k^{-1}$ weighs the two: when the state is uncertain (P_k^- large) it trusts the measurement, and when the sensor is noisy (R large) it trusts the prediction. The covariance update is written in Joseph form, which equals $(I - K_kC)P_k^-$ in exact arithmetic but stays symmetric and positive definite under rounding.

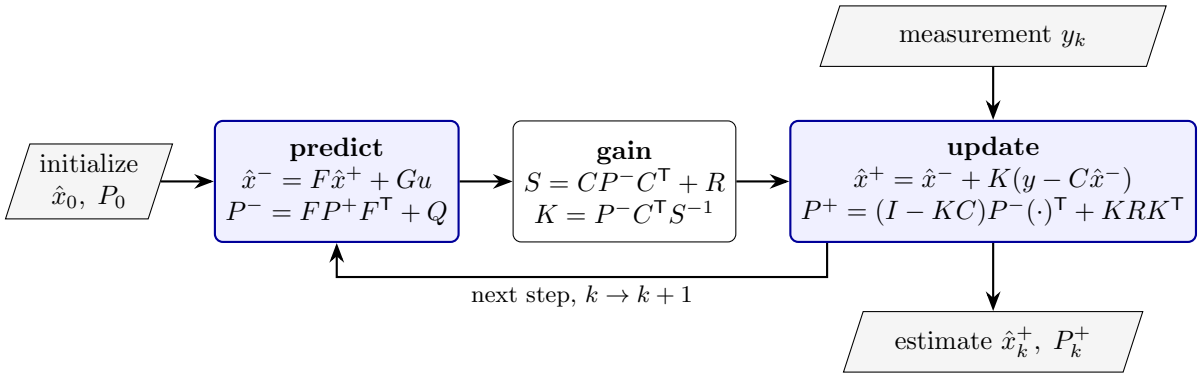


Figure 4: The Kalman filter loop. The predict step makes the covariance grow; the update step shrinks it.

Initialization. $\hat{h}_1, \hat{h}_2$ start from the first reading divided by k_c, with variance $(0.1\,\text{cm})^2$, the accuracy of one sensor reading. For the unmeasured tanks there is no reading to start from, so we deliberately start from wrong values, $\hat{h}_3(0) = 3.0$ and $\hat{h}_4(0) = 0.5\,\text{cm}$, while the true levels are 1.8 and 1.4 cm. This tests whether the filter can recover from a poor starting point. Their initial variance is $1\,\text{cm}^2$ ($\sigma = 1\,\text{cm}$), which tells the filter its starting guess may be off by about a centimetre, enough to cover these starting errors: $P_0 = \text{diag}(0.01, 0.01, 1, 1)\,\text{cm}^2$.

3.2 How the covariance terms are related

Error dynamics. Subtracting the prediction from the true model, the input terms cancel (the filter knows u exactly), and the prediction error obeys

$$e_k^- = x_k - \hat{x}_k^- = Fe_{k-1}^+ + w_{k-1}. \tag{12}$$

Because w_{k-1} is fresh noise, independent of e_{k-1}^+, the cross terms vanish in $E[e^-e^{-\mathsf{T}}]$, which gives the prediction covariance in (9).

Prediction, element by element. Rows 1 and 3 of the error equation are $e_1^- = F_{11}e_1^+ + F_{13}e_3^+ + w_1$ and $e_3^- = F_{33}e_3^+ + w_3$ (row 3 has no e_1 term because no water flows up). Squaring and multiplying, with $Q_{13} = 0$:

$$P_{11}^- = F_{11}^2P_{11}^+ + 2F_{11}F_{13}P_{13}^+ + F_{13}^2P_{33}^+ + Q_{11}, \tag{13}$$

$$P_{13}^- = F_{11}F_{33}\,P_{13}^+ + \underbrace{F_{13}F_{33}\,P_{33}^+}_{\text{creates correlation}}, \tag{14}$$

$$P_{33}^- = F_{33}^2\,P_{33}^+ + Q_{33}. \tag{15}$$

Equation (14) shows where the correlation between the h_1 and h_3 errors comes from. Even with $P_{13}^+ = 0$ (a diagonal P_0) and diagonal Q, the term $F_{13}F_{33}P_{33}^+$ is positive: the physics creates the correlation in the very first prediction. Intuitively, if $\hat{h}_3$ is too low, the filter underestimates the water draining into tank 1, so $\hat{h}_1^-$ is too low as well: the two errors lean the same way.

Gain, element by element. Because the pair (h_1, h_3) is decoupled from (h_2, h_4), the gains acting on the y_1 innovation are scalars:

$$S_{11} = k_c^2P_{11}^- + R_{11}, \qquad K_{11} = \frac{k_cP_{11}^-}{S_{11}}, \qquad K_{31} = \frac{k_cP_{31}^-}{S_{11}}, \qquad \frac{K_{31}}{K_{11}} = \frac{P_{31}^-}{P_{11}^-}. \tag{16}$$

The ratio P_{31}^-/P_{11}^- is the slope of a linear regression of the h_3 error on the h_1 error: the filter corrects $\hat{h}_1$ with the evidence, then corrects $\hat{h}_3$ by the amount the h_3 error usually moves when the h_1 error moves. If $P_{31}^- = 0$, y_1 would never correct h_3.

Update, element by element. From $P^+ = P^- - KCP^-$, i.e. $P^+_{ij} = P^-_{ij} - k_c^2 P^-_{i1} P^-_{1j}/S_{11}$:

$$P^+_{11} = P^-_{11}\,\frac{R_{11}}{S_{11}}, \qquad P^+_{13} = P^-_{13}\,\frac{R_{11}}{S_{11}}, \qquad P^+_{33} = P^-_{33} - \frac{k_c^2\,(P^-_{13})^2}{S_{11}}. \tag{17}$$

The update shrinks P_{11} by the factor R_{11}/S_{11}, but P_{33} only by an amount proportional to $(P^-_{13})^2$. A weak correlation therefore means h_3 learns very little from each measurement.

Decoupled structure. With diagonal P_0, Q and R, nothing in F, C, Q or R links the pair (h_1, h_3) to the pair (h_2, h_4). By induction on (9)–(11), $P_{12} = P_{14} = P_{23} = P_{34} = 0$ and $K_{21} = K_{41} = K_{12} = K_{32} = 0$ at every step: the four-state filter is exactly two independent two-state filters.

Steady state: the discrete algebraic Riccati equation. First, for the optimal gain the Joseph form simplifies. Expanding it,

$$P^+ = P^- - KCP^- - P^-C^{\mathsf T}K^{\mathsf T} + K\underbrace{\left(CP^-C^{\mathsf T} + R\right)}_{S}K^{\mathsf T}, \tag{18}$$

and with $K = P^-C^{\mathsf T}S^{-1}$ the last term is $KSK^{\mathsf T} = P^-C^{\mathsf T}S^{-1}CP^- = P^-C^{\mathsf T}K^{\mathsf T}$, which cancels the third. Hence

$$P^+ = (I - KC)P^- = P^- - P^-C^{\mathsf T}\big(CP^-C^{\mathsf T} + R\big)^{-1}CP^-. \tag{19}$$

Substituting this into the prediction $P^-_{k+1} = FP^+_kF^{\mathsf T} + Q$ gives a recursion in the prior covariance alone, the *Riccati difference equation*:

$$P^-_{k+1} = F\Big(P^-_k - P^-_kC^{\mathsf T}\big(CP^-_kC^{\mathsf T} + R\big)^{-1}CP^-_k\Big)F^{\mathsf T} + Q. \tag{20}$$

Every step, the bracket removes what the measurement teaches, $F(\cdot)F^{\mathsf T}$ carries the remainder forward, and Q adds new uncertainty. If the recursion settles, $P^-_{k+1} = P^-_k = P_\infty$, which gives the *discrete algebraic Riccati equation* (DARE)

$$P_\infty = F\Big(P_\infty - P_\infty C^{\mathsf T}\big(CP_\infty C^{\mathsf T} + R\big)^{-1}CP_\infty\Big)F^{\mathsf T} + Q, \qquad K_\infty = P_\infty C^{\mathsf T}\big(CP_\infty C^{\mathsf T} + R\big)^{-1}. \tag{21}$$

Because (F, C) is observable (Section 1.5) and every state is driven by noise ($Q > 0$), the DARE has a unique positive-definite solution and the recursion converges to it from any P_0. The initial guess $\hat{x}_0, P_0$ therefore affects only the transient, never the steady-state gain. Note that P_k and K_k never depend on the measured values y_k, only on F, C, Q, R and P_0, so they can even be computed before any data arrives.

4 Kalman filter results

4.1 Prior and posterior: the covariance ellipse expands and contracts

Figure 5 shows the covariance at work in the (h_1, h_3) plane. At steady state, the numbers from (13)–(17) are

$$\begin{aligned} \text{prior } P^- : &\quad P_{11} = 1.715\times10^{-3}, \quad P_{13} = 6.90\times10^{-4}, \quad P_{33} = 2.392\times10^{-2}\ \text{cm}^2, \\ \text{posterior } P^+ : &\quad P_{11} = 1.464\times10^{-3}, \quad P_{13} = 5.89\times10^{-4}, \quad P_{33} = 2.388\times10^{-2}\ \text{cm}^2. \end{aligned}$$

- **Predict expands (panel a).** Passing the posterior through F (dotted) barely changes it in 0.1 s; adding Q widens the ellipse. The h_1 standard deviation grows from 0.0383 to 0.0414 cm.

- **Update contracts (panels a, b).** The measurement shrinks P_{11} by 14.6% but P_{33} by only 0.17%. Panel (b) is taken long after the gain has converged ($t \approx 600\,\mathrm{s}$): there the expansion and the contraction balance exactly, so P returns to the same value every step. How P gets to this steady pattern from its initial value is shown in Section 4.2.

- **Shape over time (panels c, d).** At the start the ellipse is tall (P_0 allows 1 cm of error in h_3) and tilts as (14) builds up correlation: ρ_{13} peaks near 0.46 after about 2 s, while the filter is still learning about h_3. As evidence accumulates, the ellipse shrinks in h_3 and the correlation settles at $\rho_{13} = 0.108$ (prior) and 0.100 (posterior).

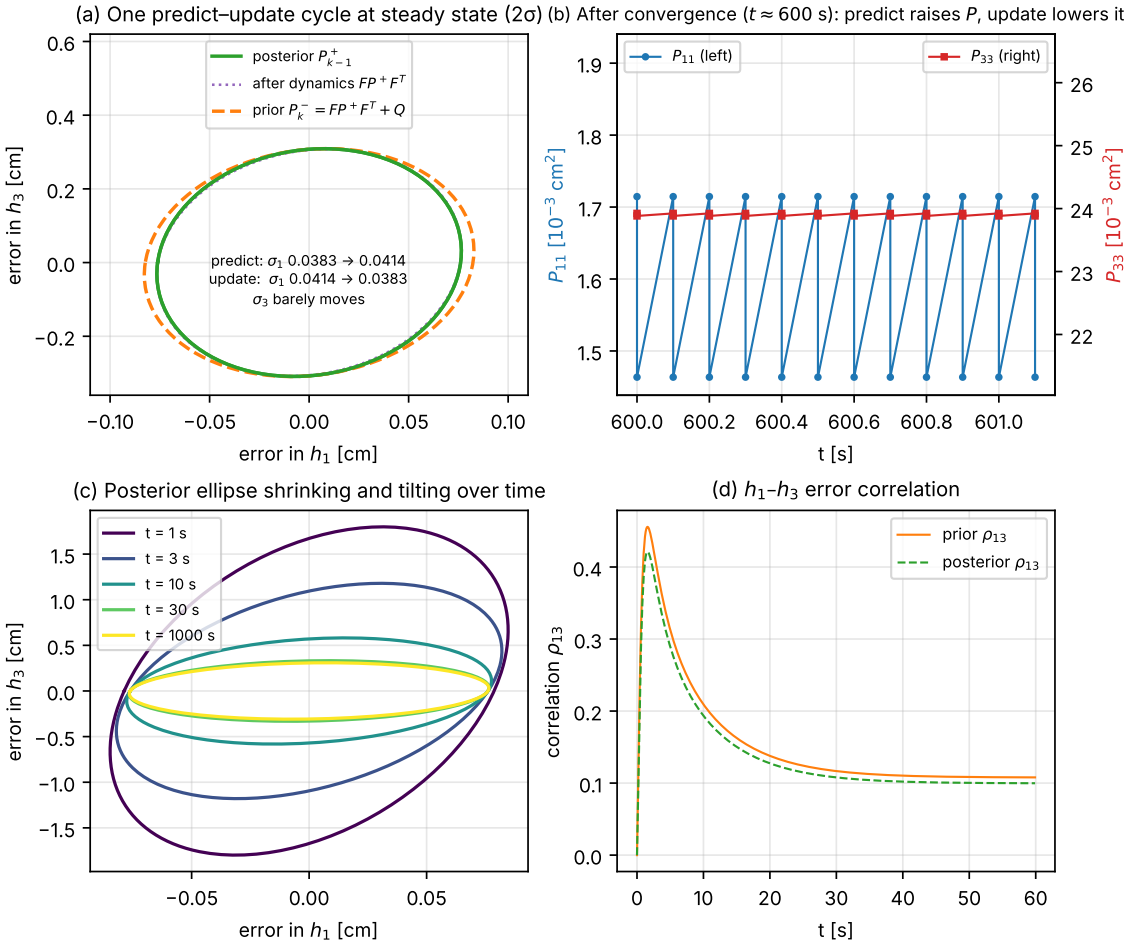


Figure 5: Covariance of the (h_1, h_3) estimation error. (a) One steady-state cycle: the prior (dashed) is wider than the posterior (solid). (b) After the gain has converged ($t \approx 600\,\mathrm{s}$): P_{11} rises at every prediction and drops back at every update by the same amount, while P_{33} barely moves. The initial descent of P_{11} from its starting value, before convergence, is shown in Figure 6. (c) Posterior 2σ ellipse during the start-up transient. (d) Correlation between the h_1 and h_3 errors.

4.2 How P_{11} converges from a poor initialization

To see the convergence of a single covariance term clearly, we start the filter with a deliberately poor initialization for tank 1, $P_{0,11} = 1\,\mathrm{cm}^2$, as if the first reading had not been used. Because the recursion (9)–(11) involves only P_0, F, Q and R and never the measured values, this isolates how the covariance itself converges (Figure 6).

- **The first update** cuts P_{11} from 1 to $0.0099\,\mathrm{cm}^2$. From (17), $P_{11}^{+} = P_{11}^{-} R_{11} / (k_c^2 P_{11}^{-} + R_{11}) \approx R_{11}/k_c^2 = 0.01$ when P_{11}^{-} is large: a single reading is as good as the sensor, $(0.1\,\mathrm{cm})^2$.

- **Then a step-by-step descent.** Each prediction raises P_{11} a little ($+Q_{11}$ plus the coupling terms in (13)); each update lowers it by more. While P_{11} is large the decreases win, so P_{11} falls quickly: P_{11}^{+} is within 25% of its steady value after 9 updates (0.9 s).
- **A slow tail.** The last part takes longer: within 10% after 4.3 s and within 1% after about 17 s. The coupling terms $2F_{11}F_{13}P_{13} + F_{13}^{2}P_{33}$ in (13) stay large while h_3 is still uncertain, so P_{11} can only settle once P_{13} and P_{33} have settled.
- **Steady state.** The up-step and down-step become equal and the same step repeats forever, between $P_{11}^{+} = 1.464 \times 10^{-3}$ and $P_{11}^{-} = 1.715 \times 10^{-3}\,\mathrm{cm}^2$, exactly the DARE values (dashed).

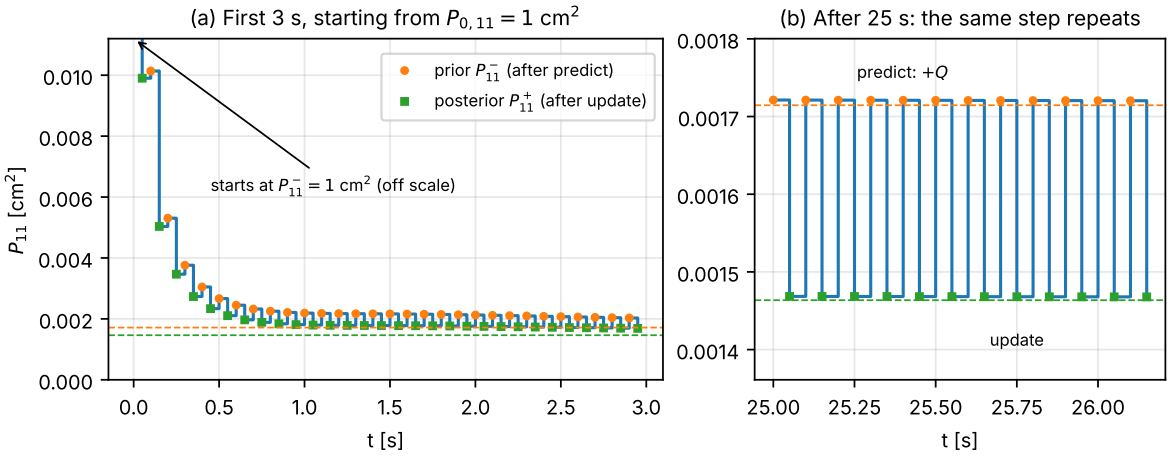


Figure 6: P_{11} starting from a poor initialization, $P_{0,11} = 1\,\mathrm{cm}^2$. Orange: prior after each prediction; green: posterior after each update; dashed: steady-state values. (a) The descent during the first 3 s. (b) After convergence, the same rise and fall repeats every step.

4.3 Convergence of the Kalman gain

Figure 7 shows every entry of K_k. The gains that move h_3 and h_4 start large (K_{31} peaks near 2.9 cm/V after about 1 s) because P_0 says little is known about those tanks, so early surprises in y_1 are blamed heavily on tank 3. As P_{33} shrinks, K_{31} settles. Within about 60 s every gain reaches the DARE value (dashed); at the end of the run the difference is 1.7×10^{-15}, i.e. machine precision:

$$K_\infty = \begin{bmatrix} 0.2927 & 0 \\ 0 & 0.2928 \\ 0.1177 & 0 \\ 0 & 0.1185 \end{bmatrix} \ \mathrm{cm/V}, \qquad k_c K_{11} = 0.146, \qquad \frac{K_{31}}{K_{11}} = 0.40.$$

So the filter accepts 14.6% of each h_1 surprise into $\hat{h}_1$, and moves $\hat{h}_3$ by 0.40 times as much as it moves $\hat{h}_1$.

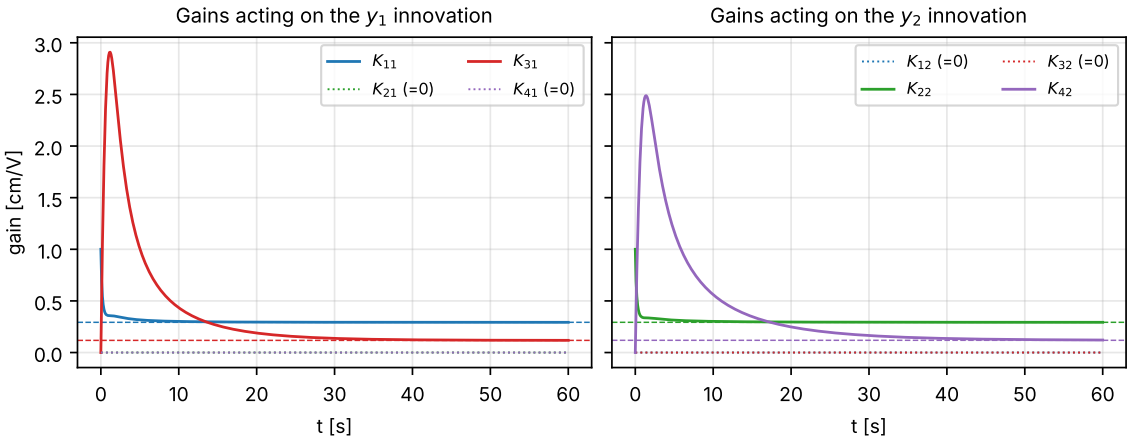


Figure 7: Kalman gain over the first 60 s. Solid: non-zero gains; dotted: gains that are exactly zero; dashed: steady-state DARE values.

4.4 Validation of the decoupled structure

The decoupling derived above is confirmed numerically (Figure 8). Over the whole run, $\max|P_{12}| = \max|P_{14}| = \max|P_{23}| = \max|P_{34}| = 0$ and $K_{21} = K_{41} = K_{12} = K_{32} = 0$ exactly. A y_1 measurement never changes $\hat{h}_2$ or $\hat{h}_4$, and a y_2 measurement never changes $\hat{h}_1$ or $\hat{h}_3$. The only off-diagonal correlations are the weak $\rho_{13} \approx 0.10$ and $\rho_{24} \approx 0.09$.

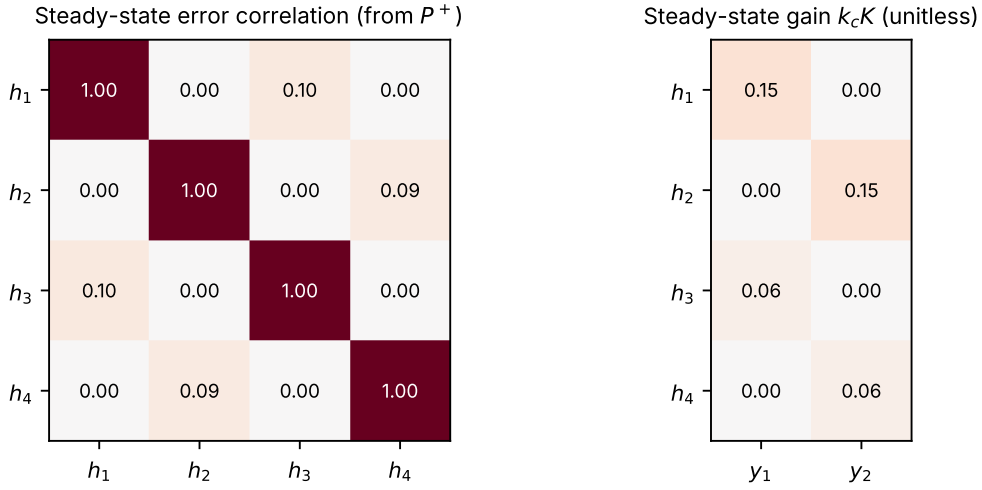


Figure 8: Steady-state error correlation matrix and gain matrix. All zeros are exact.

4.5 Filtered estimates

Figure 9 compares the measured, true and filtered levels; Table 2 summarizes the accuracy over $t \geq 100$ s. For h_1, h_2 the estimate removes most of the sensor noise (2.6× lower RMSE than the raw sensor) while still following the real, disturbance-driven wander of the level. For h_3, h_4 the wrong initial guesses are corrected within about 20–50 s, after which the estimates follow the slow trends but miss faster excursions (e.g. h_3 near 80 s). This is the observability limit from (17): a change in h_3 only shows up in y_1 after draining through the thin F_{13} link, and is masked by tank 1's own process noise.

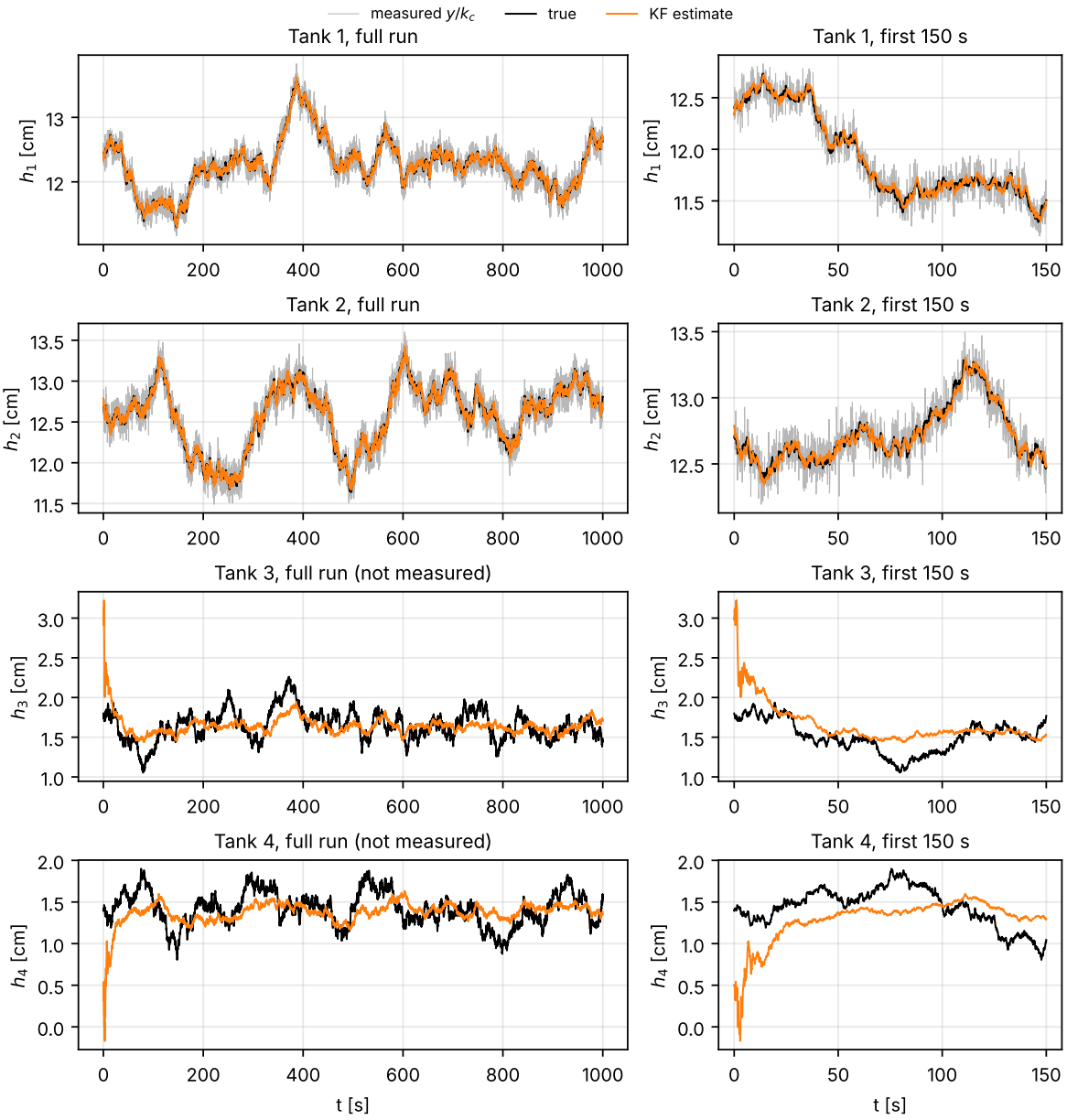


Figure 9: Measured (y/k_c, grey), true (black) and Kalman-filtered (orange) levels, over the full run (left) and the first 150 s (right).

State	Raw sensor RMSE [cm]	KF RMSE [cm]	Inside $\pm 2\sigma$ [%]	Steady σ [cm]
h_1	0.100	0.038	95.3	0.038
h_2	0.101	0.039	95.2	0.038
h_3	–	0.156	95.2	0.155
h_4	–	0.184	94.5	0.177

Table 2: Kalman filter accuracy (RMSE over $t \geq 100$ s) and consistency.

4.6 Validation with the $\pm 2\sigma$ band

Why $\pm 2\sigma$. RMSE says how large the errors are, but not whether the filter *knows* how large they are. The filter claims that its error is $e_{i,k} = x_{i,k} - \hat{x}^+_{i,k} \sim \mathcal{N}(0, \sigma^2_{i,k})$ with $\sigma_{i,k} = \sqrt{P^+_{ii,k}}$. If that claim is true,

$$\Pr\left(|e_{i,k}| \leq 2\sigma_{i,k}\right) = \operatorname{erf}\left(2/\sqrt{2}\right) = 95.45\%. \tag{22}$$

So in simulation, where the truth is known, the fraction of time the error stays inside $\pm 2\sigma$ is a direct test of the filter's covariance:

- about 95%: P is honest, so the filter is *consistent*;
- much less than 95%: the filter is *overconfident* (P too small), the dangerous case;
- much more than 95%: the filter is *underconfident* (P too large), safe but wasteful.

Convergence shows up in the same plot: the band shrinks from its initial width to a constant steady-state width, and the error stops drifting outside it. A converged filter does not have zero error. Process noise adds new uncertainty every step, so the error settles to a steady random level set by P_∞.

Result. Figure 10 shows the errors inside 95.3%, 95.2%, 95.2% and 94.5% of the time for h_1–h_4, all matching 95.45%. At $t = 0$ the h_3 and h_4 errors are large (-1.2 and $+0.9$ cm) because the filter was deliberately started from wrong values for the unmeasured tanks (Section 3.1, Initialization). Their initial variance $P_{0,33} = P_{0,44} = 1\,\text{cm}^2$ tells the filter that its starting guess may be off by about 1 cm, so the $\pm 2\sigma$ band starts wide enough to contain these starting errors. As measurements arrive, the errors decay and the bands narrow to a steady width about four times that of h_1, h_2.

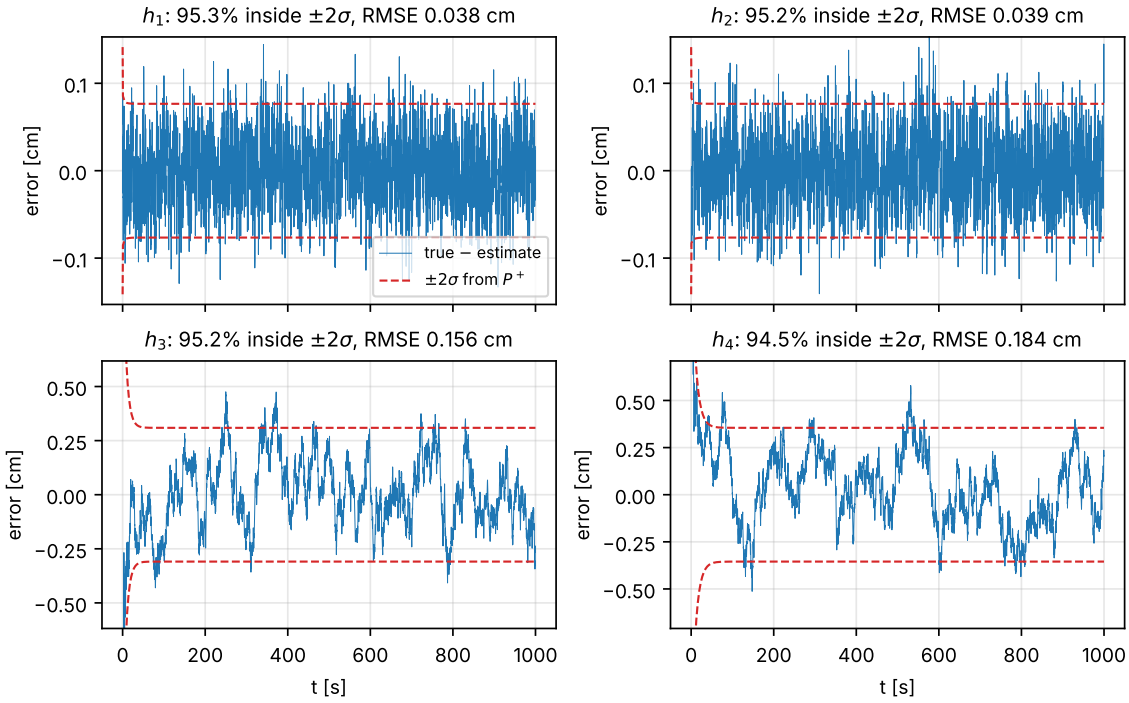


Figure 10: Estimation error (blue) with the filter's own $\pm 2\sigma$ band (red).

5 Tuning the Kalman filter

The filter's Q and R need not equal the true noise. They encode what the filter *believes*, and changing them changes the gain through (16):

- **The ratio Q/R sets the gain.** A larger Q ("my model is unreliable") or a smaller R ("my sensor is precise") raises K: the estimate follows the sensor more closely and becomes noisier. The opposite makes it smoother but slower to react.
- **The absolute size sets the claimed uncertainty P.** Two settings with the same gain can claim very different P, and only the right size gives honest $\pm 2\sigma$ bands.

- **The gain also assigns blame.** When y_1 surprises the filter, (16) splits the surprise between three suspects: the sensor (R_{11}), tank 1's own disturbance (Q_{11} through P_{11}) and tank 3 (through P_{31}). Changing one entry of Q or R changes how the blame is split.

Table 3 scales one entry at a time (the data is unchanged), and Figure 11 shows the effect on the $\pm 2\sigma$ test for h_1 (scaling Q_{11}) and h_2 (scaling R_{22}).

Filter setting	$k_c K_{11}$	K_{31} [cm/V]	RMSE h_1	RMSE h_3	h_1 in $\pm 2\sigma$ [%]	h_3 in $\pm 2\sigma$ [%]
$Q_{11} \times 0.01$	0.034	0.241	0.059	0.287	47	54
matched	0.146	0.118	0.038	0.156	95	95
$Q_{11} \times 100$	0.765	0.008	0.079	0.173	97	92
$Q_{33} \times 0.01$	0.145	0.001	0.039	0.173	95	18
$Q_{33} \times 100$	0.174	2.537	0.040	0.494	97	100
$R_{11} \times 0.01$	0.766	0.632	0.079	0.158	18	94
$R_{11} \times 100$	0.016	0.010	0.096	0.160	99	95

Table 3: Effect of scaling one entry of the filter's Q or R. RMSE in cm over $t \geq 100$ s.

- $Q_{11} \times 0.01$ ("tank 1 is never disturbed"): the h_1 gain collapses ($k_c K_{11} = 0.034$), so $\hat{h}_1$ lags the truth. Worse, every h_1 surprise is now blamed on tank 3: K_{31} doubles and the h_3 error almost doubles (0.287 cm). The filter is overconfident on both states (47% and 54% inside $\pm 2\sigma$).
- $Q_{11} \times 100$ ("tank 1 is heavily disturbed"): $\hat{h}_1$ copies the sensor ($k_c K_{11} = 0.77$), and almost nothing is blamed on tank 3 ($K_{31} = 0.008$), so $\hat{h}_3$ is effectively running on the model alone (0.173 cm, the open-loop error).
- Q_{33}: too small starves h_3 of corrections and makes the filter badly overconfident (18%); too large makes $\hat{h}_3$ soak up the sensor noise of y_1 (RMSE 0.494 cm).
- R_{11}: too small makes $\hat{h}_1$ noisy and overconfident (18%); too large makes it smooth, lagging and underconfident (99%).

In every case the matched filter (the true Q and R) gives the lowest error and honest 95% coverage, as expected: for a linear-Gaussian model the matched Kalman filter is the minimum-variance estimator.

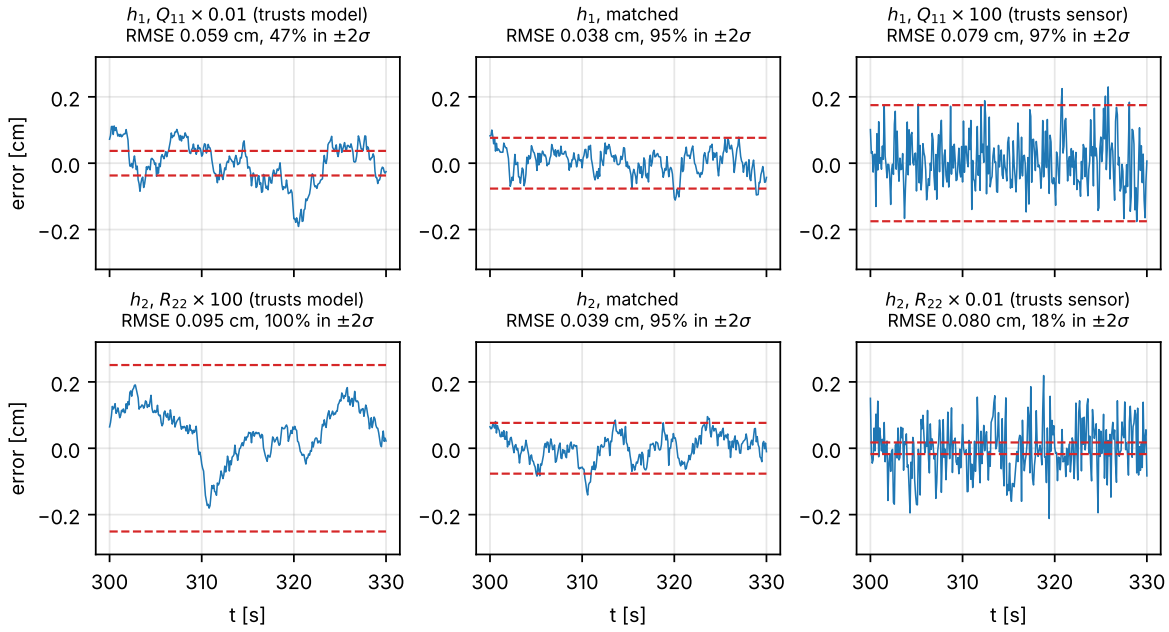


Figure 11: Error and $\pm 2\sigma$ band over a 30 s window. Left column: the filter trusts its model too much (lagging error; bands too narrow for $Q_{11} \times 0.01$, too wide for $R_{22} \times 100$). Right column: it trusts the sensor too much (noisy error).

6 The particle filter: theory

6.1 Notation

Symbol	Meaning
$y_{1:k}$	all measurements up to step k: $y_1, \ldots, y_k$
$p(x_k \mid y_{1:k})$	filtering density (posterior): what the state can be, given all measurements so far
$p(x_k \mid x_{k-1})$	transition density: where the dynamics and process noise can take the state in one step
$p(y_k \mid x_k)$	likelihood: how probable the measurement is if the state were x_k
N_p	number of particles (2000 in this work); not to be confused with N, the number of time steps
$x_k^{(i)}$	particle i: one complete candidate state $[h_1, h_2, h_3, h_4]^\mathsf{T}$, $i = 1, \ldots, N_p$
$w_k^{(i)}$	normalized weight of particle i ($\sum_i w_k^{(i)} = 1$): how well it explains the measurements
$e^{(i)} = y_k - Cx_k^{(i)}$	measurement residual of particle i
$\ell^{(i)}$	log-weight, $\log w^{(i)}$, used to avoid numerical underflow
$q(\cdot)$	proposal density: the distribution new particles are actually drawn from
N_{eff}	effective number of particles: how many particles are really contributing
u	the single random offset used by systematic resampling, $u \sim \mathcal{U}[0, 1)$
n_i	number of copies of particle i after resampling
$N_p\, w^{(i)}$	expected number of copies of particle i: a particle holding a fraction $w^{(i)}$ of the total weight gets about that fraction of the N_p new slots (e.g. $w = 0.30$ with $N_p = 8$ gives 2.4, so 2 copies)

6.2 The exact Bayesian filter

The quantity every filter approximates is the filtering density $p(x_k \mid y_{1:k})$: what the state can be, given all measurements so far. It obeys an exact two-step recursion. The *prediction* (Chapman–

Kolmogorov equation) pushes the previous belief through the dynamics,

$$p(x_k \mid y_{1:k-1}) = \int p(x_k \mid x_{k-1})\, p(x_{k-1} \mid y_{1:k-1})\, dx_{k-1}, \tag{23}$$

where the transition density follows from the model,

$$p(x_k \mid x_{k-1}) = \mathcal{N}\big(x_k;\; x_{k-1} + \Delta t\, f(x_{k-1}, u_{k-1}),\; Q\big),$$

and the *update* (Bayes' rule) reweights it by the likelihood of the new measurement,

$$p(x_k \mid y_{1:k}) = \frac{p(y_k \mid x_k)\, p(x_k \mid y_{1:k-1})}{p(y_k \mid y_{1:k-1})}, \qquad p(y_k \mid x_k) = \mathcal{N}(y_k;\; Cx_k,\; R). \tag{24}$$

Equation (23) can be read as follows. Every possible previous state x_{k-1} is moved by the non-linear dynamics to a new centre $x_{k-1} + \Delta t\, f(x_{k-1}, u_{k-1})$, and the process noise spreads it into a Gaussian blob around that centre. The new belief is the sum of all these blobs, each weighted by how likely its starting point was.

The non-linearity is what shapes the result. Because the outflow goes as $\sqrt{h}$, the dynamics do not move all states equally. Near low levels the outflow changes quickly with the level, so neighbouring states are pulled together strongly. At higher levels it changes slowly, so they are pulled together less. The low side of the belief is compressed and the high side stretched, which produces a long right tail. On top of that, a level cannot go below zero, which cuts off the lower tail altogether. Repeated every step, this uneven squeezing and stretching makes the belief skewed and shifts its average away from the equilibrium, so in general it is *not* Gaussian. The update (24) then multiplies this non-Gaussian belief by the likelihood, and neither integral has a closed form. A numerical grid is no way out either: in four dimensions, 200 cells per axis is already 1.6×10^9 cells.

Only in the special case of linear dynamics does the map move every state in the same way, so a Gaussian belief stays Gaussian and (23)–(24) reduce exactly to the Kalman filter. The Kalman filter is therefore the exact answer for a linear system and an approximation for this one. How good that approximation is depends on how strongly $\sqrt{h}$ bends over the spread of the belief, which is examined in the results.

6.3 Monte Carlo approximation

The particle filter represents the density by N_p weighted samples, the particles $x_k^{(i)}$ with weights $w_k^{(i)}$ that sum to one:

$$p(x_k \mid y_{1:k}) \approx \sum_{i=1}^{N_p} w_k^{(i)}\, \delta\big(x_k - x_k^{(i)}\big), \qquad E[g(x_k) \mid y_{1:k}] \approx \sum_{i=1}^{N_p} w_k^{(i)}\, g\big(x_k^{(i)}\big). \tag{25}$$

No shape is assumed: the cloud can be skewed, bounded or multi-modal. The price is Monte Carlo error, whose standard deviation falls only as $1/\sqrt{N_p}$. In particular the filter estimate and its covariance are

$$\hat{x}_k = \sum_i w_k^{(i)} x_k^{(i)}, \qquad P_k = \sum_i w_k^{(i)} \big(x_k^{(i)} - \hat{x}_k\big)\big(x_k^{(i)} - \hat{x}_k\big)^{\mathsf{T}}. \tag{26}$$

These are summaries for reporting, not an assumption: the cloud itself carries the full distribution.

6.4 Importance sampling and the weight update

We cannot sample the posterior directly, so particles are drawn from a *proposal* density q that we can sample, and corrected by weights $w \propto p/q$. Applied recursively (sequential importance sampling),

$$x_k^{(i)} \sim q\big(x_k \mid x_{k-1}^{(i)}, y_k\big), \qquad w_k^{(i)} \propto w_{k-1}^{(i)}\, \frac{p\big(y_k \mid x_k^{(i)}\big)\, p\big(x_k^{(i)} \mid x_{k-1}^{(i)}\big)}{q\big(x_k^{(i)} \mid x_{k-1}^{(i)}, y_k\big)}. \tag{27}$$

The **bootstrap** filter uses the dynamics themselves as the proposal, $q = p(x_k \mid x_{k-1})$: each particle is moved through the non-linear model with its own process-noise sample, and the weight update reduces to the likelihood,

$$x_k^{(i)} = \max\big(0,\; x_{k-1}^{(i)} + \Delta t\, f(x_{k-1}^{(i)}, u_{k-1}) + w^{(i)}\big), \qquad w^{(i)} \sim \mathcal{N}(0, Q), \tag{28}$$

$$w_k^{(i)} \propto w_{k-1}^{(i)}\, \exp\Big(-\tfrac{1}{2}\, e^{(i)\mathsf{T}} R^{-1} e^{(i)}\Big), \qquad e^{(i)} = y_k - C x_k^{(i)}. \tag{29}$$

This is exactly Bayes' rule (24) carried out on samples: the moved cloud is the prior, the weights are the likelihood, and the weighted cloud is the posterior. The Gaussian appears only in the noise models (and in the initial cloud $\mathcal{N}(\hat{x}_0, P_0)$), never as a shape imposed on the posterior. In practice the weights are computed in logarithms, $\ell^{(i)} = \log w_{k-1}^{(i)} - \frac{1}{2} e^{(i)\mathsf{T}} R^{-1} e^{(i)}$, and normalized as $w^{(i)} = e^{\ell^{(i)} - \max_j \ell^{(j)}} / \sum_j e^{\ell^{(j)} - \max_j \ell^{(j)}}$; otherwise every weight underflows to zero.

6.5 Degeneracy and resampling

The variance of the weights can only grow over time. Within a few dozen steps one particle carries almost all the weight and the rest contribute nothing (*degeneracy*). This is measured by the effective number of particles,

$$N_{\text{eff}} = \frac{1}{\sum_i \big(w_k^{(i)}\big)^2}, \qquad 1 \le N_{\text{eff}} \le N_p, \tag{30}$$

which equals N_p for equal weights and 1 when one particle has everything. When $N_{\text{eff}} < N_p/2$ we **resample**: draw N_p new particles from the discrete distribution $\{x^{(i)}, w^{(i)}\}$ and reset all weights to $1/N_p$. Heavy particles are copied, light ones disappear, and the next prediction's random kicks spread the copies apart again.

Systematic resampling. One random number $u \sim \mathcal{U}[0, 1)$ sets N_p evenly spaced pointers $(u + j - 1)/N_p$, $j = 1, \ldots, N_p$, on the cumulative sum of the weights; particle i is copied once for every pointer that lands in its segment (Figure 16a). Its number of copies n_i is always $\lfloor N_p w^{(i)} \rfloor$ or $\lceil N_p w^{(i)} \rceil$, so $E[n_i] = N_p w^{(i)}$ (unbiased) with the lowest variance of the simple schemes. Multinomial resampling instead makes N_p independent draws, which leaves the number of copies random and adds noise. Resampling costs $O(N_p)$.

Why only when needed. Resampling every step throws away diversity for no reason: each resample replaces the cloud with copies, so fewer distinct values remain (*sample impoverishment*). Never resampling leads to degeneracy. The threshold $N_{\text{eff}} < N_p/2$ balances the two.

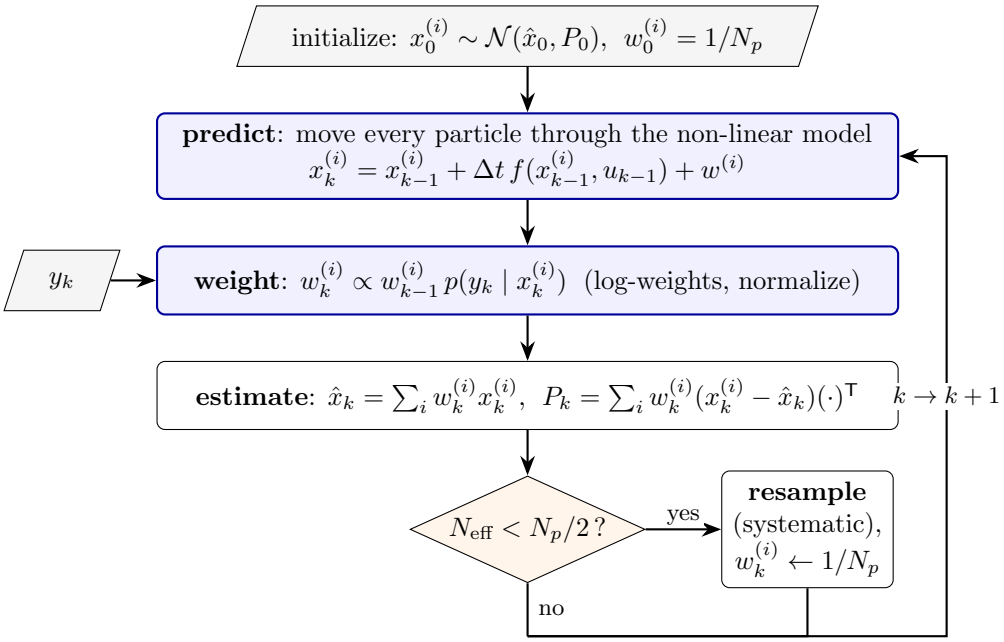


Figure 12: The bootstrap (SIR) particle filter used in this work.

6.6 Choosing the number of particles

There is no formula for N_p. The Monte Carlo error falls as $1/\sqrt{N_p}$, so four times the particles only halve it, and in weakly observed directions (here h_3, h_4) a small cloud also *underestimates* its own spread, because resampled copies spread out again only slowly. We therefore choose N_p empirically: increase it until the RMSE and the $\pm 2\sigma$ coverage stop changing across random seeds (Section 7).

7 Particle filter results

Settings. Bootstrap filter on the non-linear model, $N_p = 2000$ particles, matched Q and R, the same initial guess and P_0 as the KF, systematic resampling when $N_{\text{eff}} < N_p/2$, and log-domain weights. On the full run the filter resampled on 25% of the steps, with an average N_{eff} of 1269 out of 2000, and took about 3 s, against under 0.5 s for the KF.

7.1 Estimates and the particle cloud

Figure 13 shows the PF estimates, and Figure 14 shows the cloud of unmeasured h_3, h_4 values over the whole run. The cloud starts wide around the wrong guesses, narrows within the first 20–50 s as y_1, y_2 rule out particles whose h_3, h_4 do not fit, and then settles to a roughly constant width that moves with the truth. That settling is what convergence means for a particle filter: the cloud never shrinks to a point, because process noise keeps adding new uncertainty that the sensors have not yet seen.

The full run also shows that the large offsets seen early on (for example h_4 between 60 and 90 s) are temporary excursions, not a lasting bias. The true level drifts away from the cloud during slow disturbances and returns later, at roughly 130–150 s, 300 s, 520 s and 800 s for h_4. The KF estimate misses at exactly the same times, so this is the information limit on the unmeasured tanks, not a fault of either filter: a slow drift in h_4 shows up in y_2 only after it has drained through the thin F_{24} link. Over $t \geq 100$ s the truth lies inside the 90% band 86% (h_3) and 82% (h_4) of the time, and inside the 50% band 46% and 40% of the time: slightly below the nominal 90% and 50%, the same mild overconfidence that the $\pm 2\sigma$ test finds below.

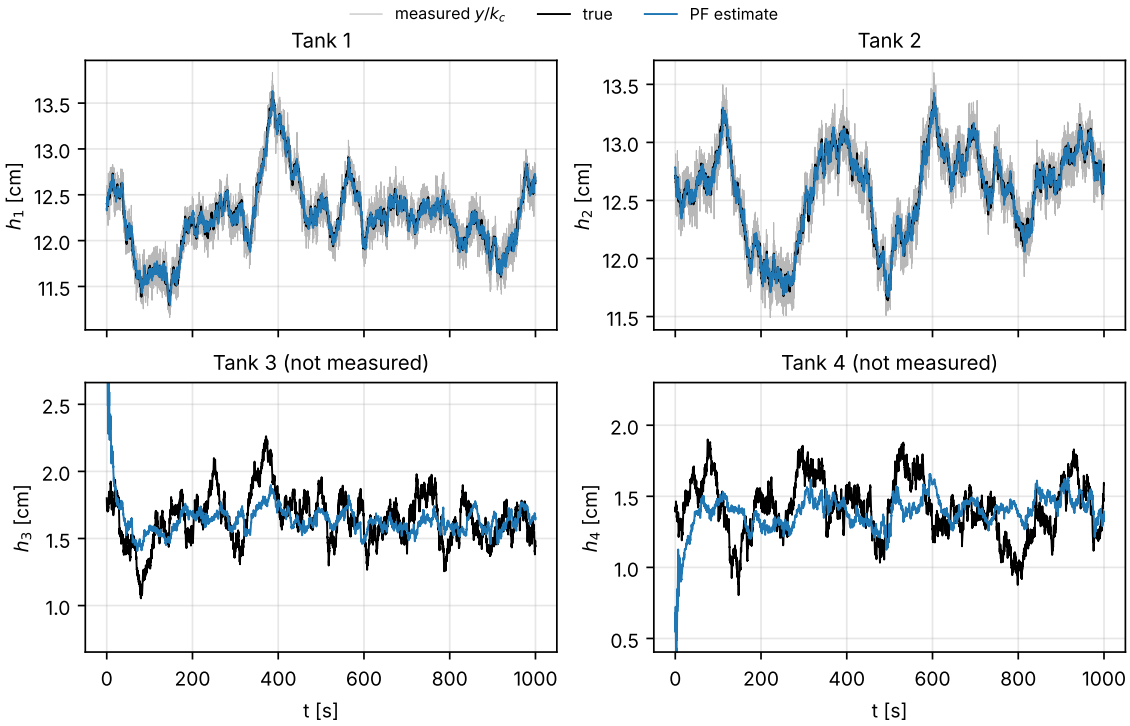


Figure 13: Particle filter estimates (blue) against the true (black) and measured (grey) levels.

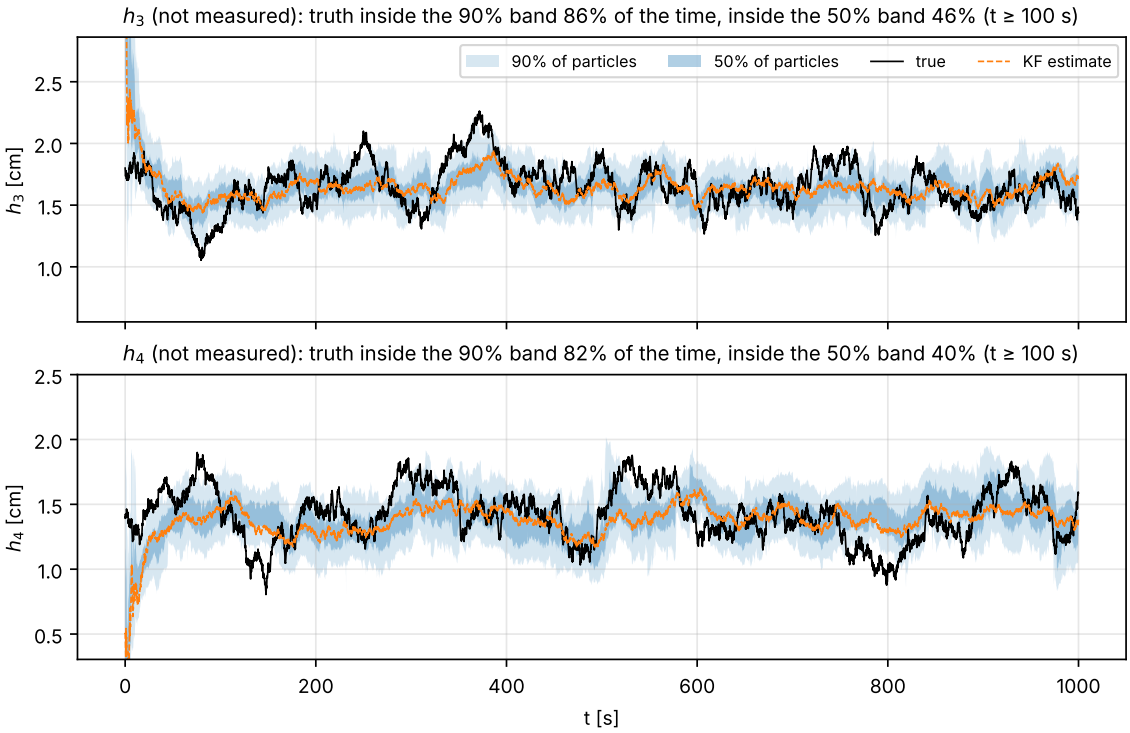


Figure 14: The particle cloud for the unmeasured tanks over the full run: shaded bands contain 50% and 90% of the particle weight. The KF estimate (dashed) is shown for comparison.

Figure 15 shows the cloud in the (h_1, h_3) plane with the KF's 2σ ellipse. The PF has no covariance matrix and no gain, yet its cloud reproduces the KF's ellipse, including the slight tilt of the weak h_1–h_3 correlation. The correlation emerges purely from which particles survive: a particle with h_3 too high drains too much water into tank 1, so its h_1 soon disagrees with y_1, its weight

drops, and it dies at resampling. At $t = 100\,\mathrm{s}$ the true state lies outside both the cloud and the ellipse; a 2σ region should miss about 5% of the time, and the $\pm 2\sigma$ statistics below judge the filter better than one snapshot.

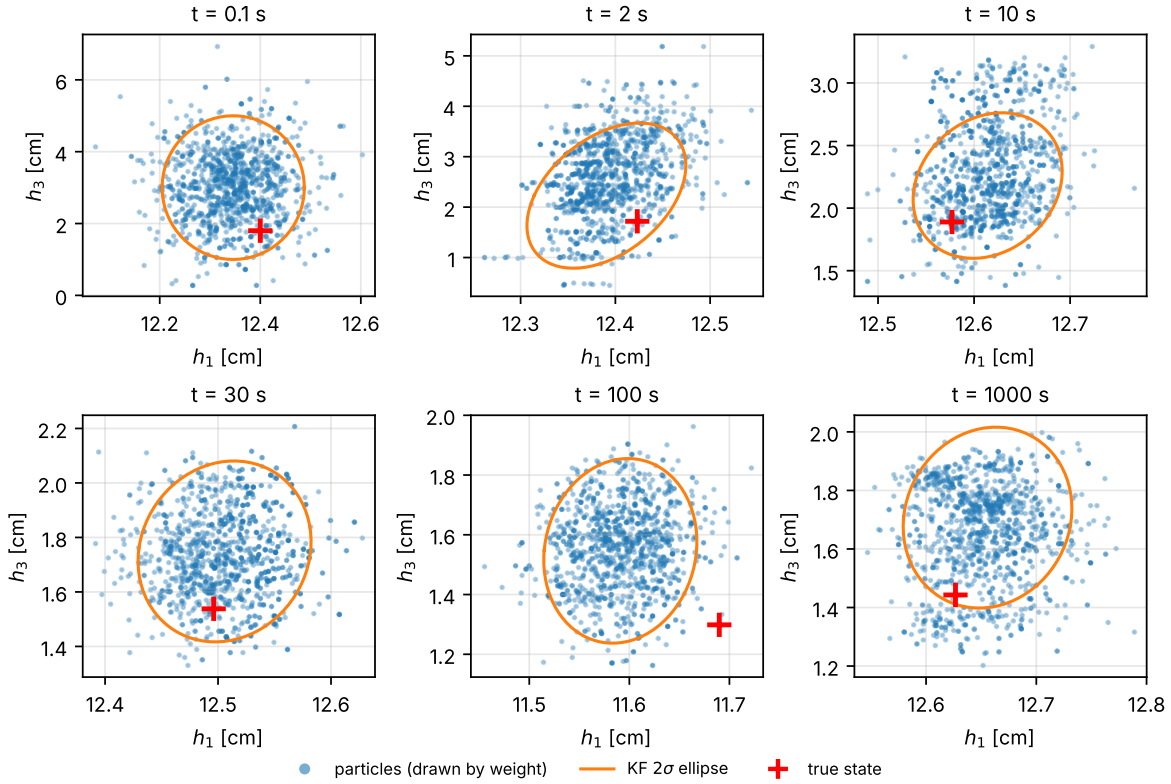


Figure 15: Particle cloud in the (h_1, h_3) plane (dots drawn in proportion to weight), the KF's 2σ ellipse and the true state.

7.2 Resampling in practice

Figure 16 makes resampling visible. Panel (a) is the systematic scheme on 8 particles: particles 2, 4 and 5 (weights 0.30, 0.18, 0.25) each receive two pointers and are copied twice, particles 1 and 7 are copied once, and particles 3, 6 and 8 receive none. The number of copies is always $\lfloor N_p w \rfloor$ or $\lceil N_p w \rceil$: particle 2, with $N_p w = 8 \times 0.30 = 2.4$, gets two. Panel (b) shows N_{eff} over 4 s of the real run: each new measurement makes the weights more uneven and pulls N_{eff} down; when it crosses $N_p/2$ the cloud is resampled (red bands) and the weights are equalized. Panel (c) shows one real resampling step. Each dot is a particle, placed by its h_1 value and its weight; the weight has built up over the steps since the last resample. The heaviest particles receive four or more copies, and 49% of the particles receive none.

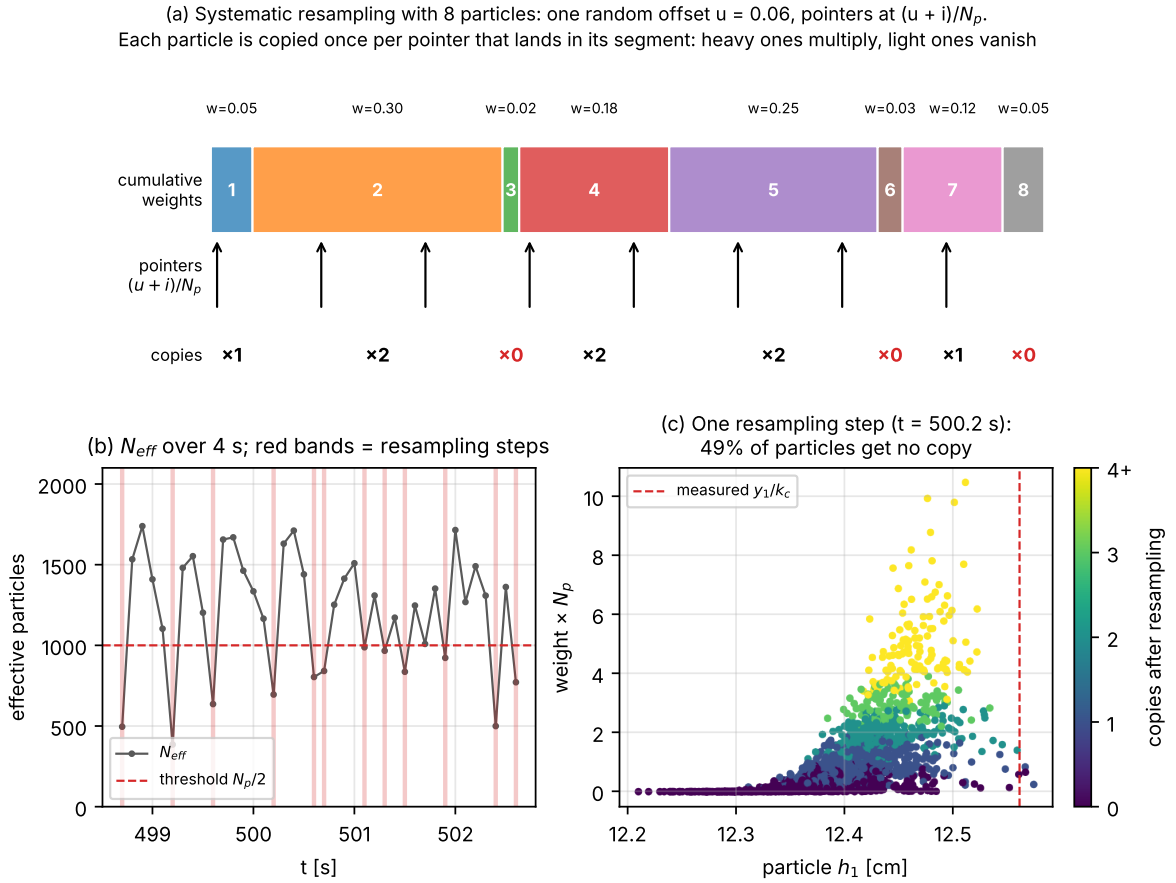


Figure 16: Resampling. (a) Systematic resampling on a toy example. (b) Effective number of particles in the real run. (c) One real resampling step: weight of each particle against its h_1 value, coloured by how many copies it received.

7.3 Validation and number of particles

Figure 17 applies the same $\pm 2\sigma$ test, using the cloud's own spread as σ. For h_1, h_2 the PF is consistent (95.3%, 95.0%). For h_3, h_4 it is somewhat overconfident (92.5%, 86.5%). This is sample impoverishment: after each resample the cloud consists of families of copies, and in directions the sensors barely constrain, these families mix back slowly, so the cloud is a little too narrow.

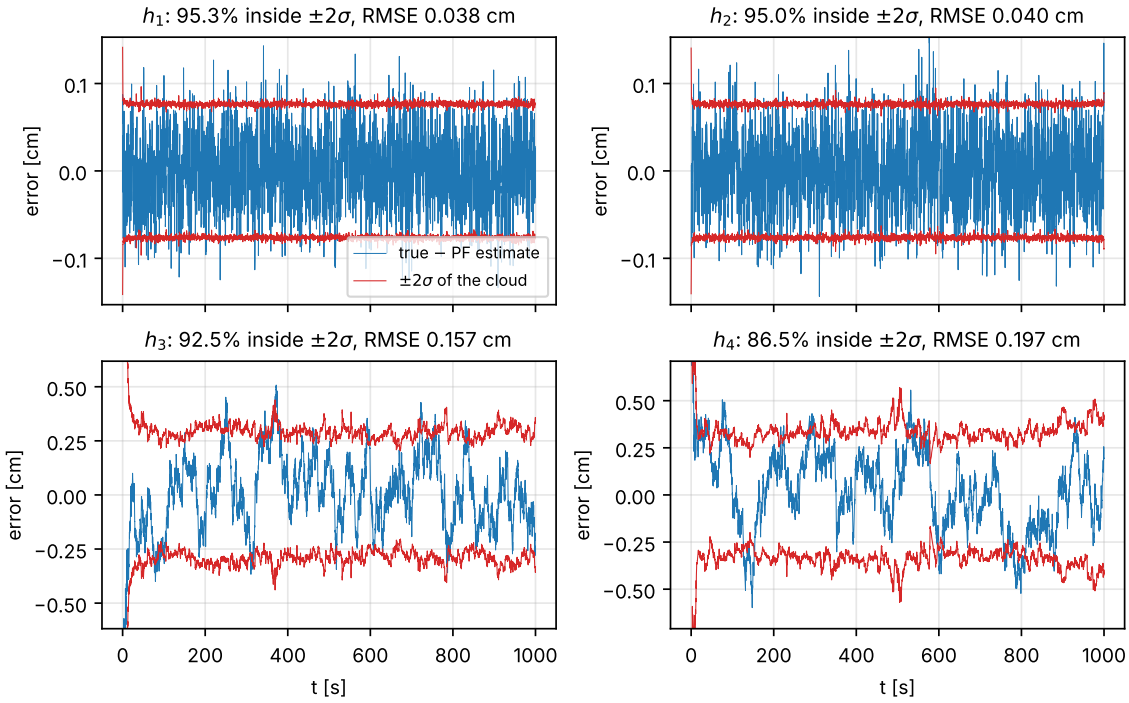


Figure 17: Particle filter error with the $\pm 2\sigma$ band of the particle cloud.

Figure 18 and Table 4 show how this depends on N_p (three random seeds per value). h_1, h_2 are accurate even with 100 particles. h_3, h_4 need more: their RMSE levels off at about 1000–3000 particles, within about 5% of the KF, and their coverage climbs from about 60% to 90–93% without quite reaching 95%. We use $N_p = 2000$ as the balance between accuracy and run time. Common remedies for the remaining overconfidence are more particles or *roughening* (a small random jitter added after each resample).

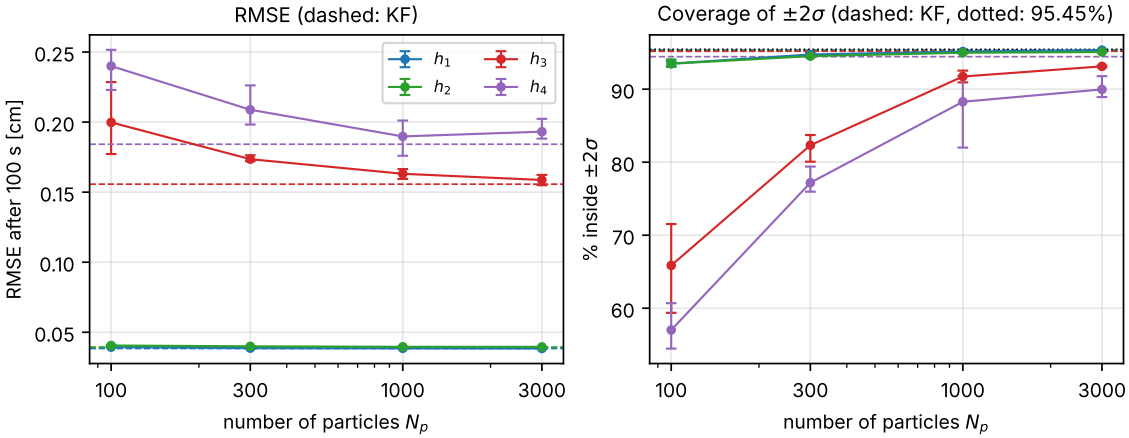


Figure 18: Effect of the number of particles. Points: mean over three seeds; bars: range; dashed: Kalman filter.

Particles	RMSE h_3	RMSE h_4	h_3 in $\pm 2\sigma$ [%]	h_4 in $\pm 2\sigma$ [%]	Run time [s]
100	0.200	0.240	65.9	57.0	0.7
300	0.174	0.209	82.3	77.2	1.1
1000	0.163	0.190	91.7	88.3	1.8
3000	0.159	0.193	93.1	90.0	4.9
KF	0.156	0.184	95.2	94.5	0.28

Table 4: Particle count study (means over three seeds; RMSE in cm over $t \geq 100\,\mathrm{s}$; run time per filter run).

7.4 Kalman filter vs particle filter

State	KF RMSE [cm]	PF RMSE [cm]	KF in $\pm 2\sigma$ [%]	PF in $\pm 2\sigma$ [%]
h_1	0.038	0.038	95.3	95.3
h_2	0.039	0.040	95.2	95.0
h_3	0.156	0.157	95.2	92.5
h_4	0.184	0.197	94.5	86.5

Table 5: Kalman filter vs particle filter ($N_p = 2000$).

The two filters reach almost the same accuracy (Table 5, Figure 19), and their errors move together, because both are limited by the same information. The PF's main advantage, using the exact nonlinear model and allowing any shape of distribution, does not pay off here. At this noise level the tanks stay within about $\pm 10\%$ of equilibrium, where $\sqrt{h}$ is almost a straight line, so the true filtering distribution is very nearly Gaussian and the Kalman filter is effectively optimal. The PF matches it at about seven times the computing time (and about 2000 model evaluations per step instead of a few 4×4 matrix products), and is slightly overconfident on the unmeasured tanks.

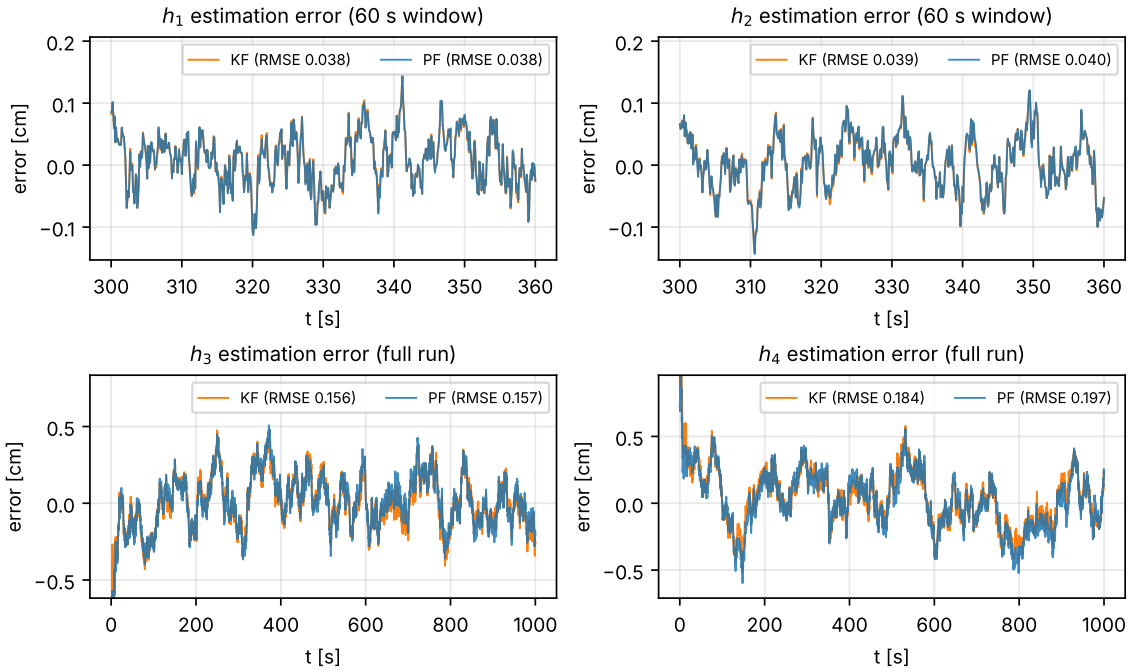


Figure 19: Estimation errors of the Kalman filter and the particle filter: a 60 s window for h_1, h_2, where the two errors lie almost on top of each other, and the full run for h_3, h_4.

8 Conclusions

- The quadruple-tank filter splits into two independent two-state filters, (h_1, h_3) and (h_2, h_4). The unmeasured tanks are observable only through the thin couplings $F_{13} = 0.0044$ and $F_{24} = 0.0033$.
- The covariance relations explain the results. The prediction creates the h_1–h_3 correlation through $F_{13}F_{33}P_{33}$; the gain $K_{31} = K_{11}P_{31}/P_{11}$ passes h_1 evidence on to h_3; and because the correlation is weak ($\rho \approx 0.1$), each update shrinks P_{11} by 14.6% but P_{33} by only 0.17%.
- The Kalman filter reduces the h_1, h_2 error 2.6× below the sensor's and estimates h_3, h_4 to 0.16 and 0.18 cm. Its gain converges to the Riccati solution within about a minute, and its errors stay inside $\pm 2\sigma$ about 95% of the time, so its uncertainty is honest.
- Tuning works through the ratio Q/R (the gain) and the absolute size (the claimed uncertainty). Misplaced blame can hurt a state that was not tuned: $Q_{11} \times 0.01$ almost doubles the h_3 error.
- The particle filter, using the exact non-linear model, matches the Kalman filter's accuracy with about 2000 particles, but costs far more and is slightly overconfident on h_3, h_4 because of sample impoverishment. A particle filter earns its cost only when the non-linearity or the noise is large enough to make the distribution clearly non-Gaussian, which is not the case at this operating point.